



CS2009

DESIGN AND ANALYSIS OF ALGORITHMS

Dr. Hashim Yasin

**National University of Computer
and Emerging Sciences,
Faisalabad, Pakistan.**

ANALYSIS OF RECURSIVE ALGORITHMS



Recurrence Relation

3

- A recurrence relation, $T(n)$, is a recursive function of an integer variable n .
- Like all recursive functions, it has one or more recursive cases and one or more base cases.

For example,

$$T(n) = \begin{cases} a & \text{if } n = 1 \\ 2T(n/2) + bn + c & \text{if } n > 1 \end{cases}$$

Recurrence Relation

4

$$T(n) = \begin{cases} a & \text{if } n = 1 \\ 2T(n/2) + bn + c & \text{if } n > 1 \end{cases}$$

- The portion of the definition that does not contain T is called the **base case** of the recurrence relation;
- the portion that contains T is called the **recurrent or recursive case**.

Recurrence Relation

5

Example 1:

Write the recurrence relation for the following method:

```
void f (int n) {  
    if (n > 0) {  
        cout << n << endl;  
        f(n-1);  
    }  
}
```

1. The **base case is reached when $n == 0$** . The method performs one comparison. Thus, the number of operations when $n == 0$, $T(0)$, is some constant a .

Recurrence Relation

6

Example 1:

Write the recurrence relation for the following method:

```
void f (int n) {  
    if (n > 0) {  
        cout << n << endl;  
        f(n-1);  
    }  
}
```

2. When $n > 0$, the method performs two basic operations and then calls itself, using ONE recursive call, with a parameter $n - 1$.

Recurrence Relation

7

Example 1:

Write the recurrence relation for the following method:

```
void f (int n) {  
    if (n > 0) {  
        cout << n << endl;  
        f(n-1);  
    }  
}
```

□ Therefore, the recurrence relation is:

$$T(n) = a \quad \text{if } n = 0$$

$$T(n) = b + T(n - 1) \quad \text{if } n > 0$$

Recurrence Relation

8

Example 1:

Write the recurrence relation for the following method:

```
void f (int n) {  
    if (n > 0) {  
        cout << n << endl;  
        f(n-1);  
    }  
}
```

□ Therefore, the recurrence relation is:

$T(0) = a$ for some constant a

$T(n) = b + T(n - 1)$ for some constant b

Recurrence Relation

9

Example 2:

Write the recurrence relation for the following method:

```
int g(int n) {  
    if (n == 1)  
        return 2;  
    else  
        return 3 * g(n / 2) + g(n / 2) + 5;  
}
```

1. The **base case is reached when $n == 1$** . The method performs one comparison and one return statement. Therefore, $T(1)$, is some constant c .

Recurrence Relation

10

Example 2:

Write the recurrence relation for the following method:

```
int g(int n) {  
    if (n == 1)  
        return 2;  
    else  
        return 3 * g(n / 2) + g(n / 2) + 5;  
}
```

2. When $n > 1$, the method performs **TWO** recursive calls, each with the parameter $n / 2$, and some constant for # of basic operations.

Recurrence Relation

11

Example 2:

Write the recurrence relation for the following method:

```
int g(int n) {  
    if (n == 1)  
        return 2;  
    else  
        return 3 * g(n / 2) + g(n / 2) + 5;  
}
```

□ Hence, the recurrence relation is:

$$T(n) = c \quad \text{if } n = 1$$

$$T(n) = b + 2T(n / 2) \quad \text{if } n > 1$$

Recurrence Relation

12

Example 2:

Write the recurrence relation for the following method:

```
int g(int n) {  
    if (n == 1)  
        return 2;  
    else  
        return 3 * g(n / 2) + g(n / 2) + 5;  
}
```

□ Hence, the recurrence relation is:

$T(1) = c$ for some constant c

$T(n) = b + 2T(n / 2)$ for some constant b

Recurrence Relation

13

Example 3:

Write the recurrence relation for the following method:

```
long fibonacci (int n) { // Recursively calculates Fibonacci number
    if( n == 1 || n == 2)
        return 1;
    else
        return fibonacci(n - 1) + fibonacci(n - 2);
}
```

1. The base case is reached when $n == 1$ or $n == 2$. The method performs one comparison and one return statement. Therefore, each of $T(1)$ and $T(2)$ is some constant c .

Recurrence Relation

14

Example 3:

Write the recurrence relation for the following method:

```
long fibonacci (int n) { // Recursively calculates Fibonacci number
    if( n == 1 || n == 2)
        return 1;
    else
        return fibonacci(n - 1) + fibonacci(n - 2);
}
```

2. When $n > 2$, the method performs **TWO** recursive calls, one with the parameter $n - 1$, another with parameter $n - 2$, and some constant for # of basic operations.

Recurrence Relation

15

Example 3:

Write the recurrence relation for the following method:

```
long fibonacci (int n) { // Recursively calculates Fibonacci number
    if( n == 1 || n == 2)
        return 1;
    else
        return fibonacci(n - 1) + fibonacci(n - 2);
}
```

□ Hence, the recurrence relation is:

$$T(n) = c \quad \text{if } n = 1 \text{ or } n = 2$$

$$T(n) = T(n - 1) + T(n - 2) + b \quad \text{if } n > 2$$

Recurrence Relation

16

- To solve a recurrence relation $T(n)$ we need to derive a form of $T(n)$ that is not a recurrence relation.
 - ▣ Such a form is called a **closed form** of the recurrence relation.
- There are **five methods to solve recurrence relations** that represent the running time of recursive methods:
 - Iteration method (*unrolling and summing*)
 - Substitution method (*Guess the solution and verify by induction*)
 - Recursion tree method
 - Master theorem (Master method)
 - Using Generating functions or Characteristic equations

ITERATION METHOD



Iteration Method

18

□ Main Steps:

- **Expand** the recurrence
 - Express the **expansion as a summation** by plugging the recurrence back into itself until you see a pattern.
 - **Evaluate the summation**
- In evaluating the summation one or more of the following summation formulae may be used:

Review on Summations

- **Constant Series:** For integers a and b , $a \leq b$,

$$\sum_{i=a}^b 1 = b - a + 1$$

- **Linear Series (Arithmetic Series):** For $n \geq 0$,

$$\sum_{i=1}^n i = 1 + 2 + \cdots + n = \frac{n(n+1)}{2}$$

- **Quadratic Series:** For $n \geq 0$,

$$\sum_{i=1}^n i^2 = 1^2 + 2^2 + \cdots + n^2 = \frac{n(n+1)(2n+1)}{6}$$

Review on Summations

- **Cubic Series:** For $n \geq 0$,

$$\sum_{i=1}^n i^3 = 1^3 + 2^3 + \cdots + n^3 = \frac{n^2(n+1)^2}{4}$$

- **Geometric Series:** For real $x \neq 1$,

$$\sum_{k=0}^n x^k = 1 + x + x^2 + \cdots + x^n = \frac{x^{n+1} - 1}{x - 1}$$

Review on Summations

□ **Linear-Geometric Series:** For $n \geq 0$, real $c \neq 1$,

$$\sum_{i=1}^n ic^i = c + 2c^2 + \cdots + nc^n = \frac{-(n+1)c^{n+1} + nc^{n+2} + c}{(c-1)^2}$$

Review on Summations

Harmonic Series:

$$\sum_{k=1}^n \frac{1}{k} = 1 + \frac{1}{2} + \dots + \frac{1}{n} \approx \ln n$$

Others:

$$\sum_{k=1}^n \lg k \approx n \lg n$$

$$\sum_{k=0}^{n-1} c = cn.$$

$$\sum_{k=0}^{n-1} \frac{1}{2^k} = 2 - \frac{1}{2^{n-1}}$$

$$\sum_{k=1}^n k^2 = \frac{n(n+1)(2n+1)}{6}$$

$$\sum_{k=0}^n k(k+1) = \frac{n(n+1)(n+2)}{3}$$

ITERATION METHOD
EXAMPLE 1
FACTORIAL METHOD

Example 1

24

- Form and solve the recurrence relation for the running time of **factorial method** and hence determine its big-O complexity:

```
long factorial (int n) {  
    if (n == 0)  
        return 1;  
    else  
        return n * factorial (n - 1);  
}
```

- The recurrence relation is:

$$T(n) = c \quad \text{if } n = 0$$

$$T(n) = b + T(n - 1) \quad \text{if } n > 0$$

Example 1

25

- Form and solve the recurrence relation for the running time of **factorial method** and hence determine its big-O complexity:

```
long factorial (int n) {  
    if (n == 0)  
        return 1;  
    else  
        return n * factorial (n - 1);  
}
```

- The recurrence relation is:

$$T(0) = c \quad \text{for some constant } c$$

$$T(n) = b + T(n - 1) \quad \text{for some constant } b$$

Example 1

$$\begin{array}{ll} T(n) = c & \text{if } n = 0 \\ T(n) = b + T(n - 1) & \text{if } n > 0 \end{array}$$

26

$$T(0) = c$$

(1)

$$T(n) = b + T(n - 1)$$

(2)

$$= b + b + T(n - 2)$$

by substitution $T(n - 1)$ in (2)

$$= b + b + b + T(n - 3)$$

by substitution $T(n - 2)$ in (2)

$$T(n) = b + T(n - 1)$$

Now substitute $T(n - 1)$ from the same formula:

$$T(n - 1) = b + T(n - 2)$$

So,

$$T(n) = b + (b + T(n - 2)) = 2b + T(n - 2)$$

Now substitute $T(n - 2)$:

$$T(n - 2) = b + T(n - 3)$$

So,

$$T(n) = 2b + (b + T(n - 3)) = 3b + T(n - 3)$$

Example 1

$$\begin{array}{ll} T(n) = c & \text{if } n = 0 \\ T(n) = b + T(n - 1) & \text{if } n > 0 \end{array}$$

27

$$T(0) = c \quad (1)$$

$$T(n) = b + T(n - 1) \quad (2)$$

$$= b + b + T(n - 2) \quad \text{by substitution } T(n - 1) \text{ in (2)}$$

$$= b + b + b + T(n - 3) \quad \text{by substitution } T(n - 2) \text{ in (2)}$$

...

$$= kb + T(n - k)$$

The base case is reached when $n - k = 0 \rightarrow k = n$, we then have:

$$T(n) = bn + T(n - n)$$

$$= bn + T(0)$$

$$= bn + c$$

Therefore,
the method factorial is $O(n)$

ITERATION METHOD
EXAMPLE 2
BINARY SEARCH



Example 2

29

- Form and solve the recurrence relation for the running time of **Binary Search** and hence determine its big-O complexity:

```
int binarySearch (int target, int[] array,
                  int low, int high) {
    if (low > high)
        return -1;
    else {
        int middle = (low + high)/2;
        if (array[middle] == target)
            return middle;
        else if (array[middle] < target)
            return binarySearch(target, array, middle + 1, high);
        else
            return binarySearch(target, array, low, middle - 1);
    }
}
```

Example 2

30

```
int binarySearch (int target, int[] array,
                  int low, int high) {
    if (low > high)
        return -1;
    else {
        int middle = (low + high)/2;
        if (array[middle] == target)
            return middle;
        else if (array[middle] < target)
            return binarySearch(target, array, middle + 1, high);
        else
            return binarySearch(target, array, low, middle - 1);
    }
}
```

□ The recurrence relation for running time of the method is:

$T(n) = a$ if $n = 1$ (one element array)

$T(n) = T(n / 2) + b$ if $n > 1$

Example 2

$$\begin{aligned} T(n) &= a && \text{if } n = 1 \\ T(n) &= T(n/2) + b && \text{if } n > 1 \end{aligned}$$

31

Expanding:

$$T(1) = a \quad (1)$$

$$T(n) = T(n/2) + b \quad (2)$$

$$= [T(n/2^2) + b] + b = T(n/2^2) + 2b \quad \text{by substitution } T(n/2) \text{ in } (2)$$

$$= [T(n/2^3) + b] + 2b = T(n/2^3) + 3b \quad \text{by substitution } T(n/2^2) \text{ in } (2)$$

$$T(n) = T\left(\frac{n}{2}\right) + b$$

Now substitute $T(n/2)$ using the same formula:

$$T\left(\frac{n}{2}\right) = T\left(\frac{n}{2^2}\right) + b$$

So,

$$T(n) = \left(T\left(\frac{n}{2^2}\right) + b\right) + b = T\left(\frac{n}{2^2}\right) + 2b$$

Now substitute $T(n/2^2)$:

$$T\left(\frac{n}{2^2}\right) = T\left(\frac{n}{2^3}\right) + b$$

So,

$$T(n) = T\left(\frac{n}{2^3}\right) + 3b$$

Example 2

$$\begin{array}{ll} T(n) = a & \text{if } n = 1 \\ T(n) = T(n / 2) + b & \text{if } n > 1 \end{array}$$

32

Expanding:

$$T(1) = a \quad (1)$$

$$T(n) = T(n / 2) + b \quad (2)$$

$$= [T(n / 2^2) + b] + b = T(n / 2^2) + 2b \quad \text{by substitution } T(n/2) \text{ in } (2)$$

$$= [T(n / 2^3) + b] + 2b = T(n / 2^3) + 3b \quad \text{by substitution } T(n/2^2) \text{ in } (2)$$

$$= \dots\dots\dots$$

$$= T(n / 2^k) + kb$$

The base case is reached when $n / 2^k = 1 \Rightarrow n = 2^k \Rightarrow k = \log_2 n$, we have:

$$T(n) = T(1) + b \log_2 n$$

$$= a + b \log_2 n$$

Therefore,
Recursive Binary Search is **$O(\log n)$**

ITERATION METHOD

EXAMPLE 3

RECURSIVE SELECTION SORT



Example 3

34

- Solve the following recurrence relation for the running time of **Recursive Selection Sort** and hence determine its big-O complexity:
- The recurrence relation for running time of the **Recursive Selection Sort** method is:

$$T(0) = a$$

$$T(n) = T(n - 1) + n + c \quad \text{if } n > 0$$

Example 3

$$T(0) = a$$

$$T(n) = T(n - 1) + n + c \quad \text{if } n > 0$$

35

$$T(0) = a \quad (1)$$

$$T(n) = T(n - 1) + n + c \quad \text{if } n > 0 \quad (2)$$

$$= [T(n-2) + (n-1) + c] + n + c = T(n-2) + (n-1) + n + 2c$$

by substitution $T(n-1)$ in (2)

$$= [T(n-3) + (n-2) + c] + (n-1) + n + 2c = T(n-3) + (n-2) + (n-1) + n + 3c$$

by substitution $T(n-2)$ in (2)

$$T(n) = T(n - 1) + n + c$$

Now substitute $T(n - 1)$ using the same formula:

$$T(n - 1) = T(n - 2) + (n - 1) + c$$

So,

$$T(n) = [T(n - 2) + (n - 1) + c] + n + c$$

$$= T(n - 2) + (n - 1) + n + 2c$$

Now substitute $T(n - 2)$:

$$T(n - 2) = T(n - 3) + (n - 2) + c$$

So,

$$T(n) = [T(n - 3) + (n - 2) + c] + (n - 1) + n + 2c$$

$$= T(n - 3) + (n - 2) + (n - 1) + n + 3c$$

Example 3

$$T(0) = a$$

$$T(n) = T(n - 1) + n + c \quad \text{if } n > 0$$

36

$$T(0) = a \quad (1)$$

$$T(n) = T(n - 1) + n + c \quad \text{if } n > 0 \quad (2)$$

$$= [T(n-2) + (n-1) + c] + n + c = T(n-2) + (n-1) + n + 2c$$

by substitution $T(n-1)$ in (2)

$$= [T(n-3) + (n-2) + c] + (n-1) + n + 2c = T(n-3) + (n-2) + (n-1) + n + 3c$$

by substitution $T(n-2)$ in (2)

$$= T(n-4) + (n-3) + (n-2) + (n-1) + n + 4c$$

$$= \dots\dots$$

$$= T(n-k) + (n-k + 1) + (n-k + 2) + \dots\dots + n + kc$$

The base case is reached when $n - k = 0 \rightarrow k = n$, we then have :

$$= T(0) + (1) + (2) + \dots\dots + n + nc$$

Example 3

$$T(0) = a$$

$$T(n) = T(n - 1) + n + c \quad \text{if } n > 0$$

37

The base case is reached when $n - k = 0 \rightarrow k = n$, we then have :

$$= T(0) + (1) + (2) + \dots + n + nc$$

$$T(n) = T(0) + 1 + 2 + \dots + n + nc$$

$$= a + \sum_{i=1}^n i + cn$$

$$= a + \left(\frac{n(n+1)}{2} \right) + cn$$

$$= \frac{n^2}{2} + \left(c + \frac{1}{2} \right) n + a$$

Arithmetic Series:

$$\sum_{i=1}^n i = 1 + 2 + \dots + n = \frac{n(n+1)}{2}$$

Therefore,
Recursive Selection Sort is **$O(n^2)$**

Acknowledgement

38

Introduction to Algorithms

Thomas H. Cormen, Charles E. Leiserson,
Ronald L. Rivest, and Clifford Stein